Portable Evidence Runtime

Co-Packaging Memory, Rules, Verification, and Software for Offline Recovery

Mitchell D. McPhetridge
Independent Researcher
11 November 2026
Research preprint and executable reference experiment · Version 0.2

Publication status: Public research prototype; not peer reviewed. The results below are local engineering checks, not an independent validation study.

Abstract

Persistent knowledge is commonly described as something a software service remembers, a provider hosts, or a model retrieves. Those are deployment arrangements, not necessary properties of the knowledge itself. This paper asks a more fundamental engineering question: **If memory, rules, verification, and software can travel together, why require a particular notebook, cloud account, or AI provider to reconstruct the system?** I propose a *portable evidence runtime*: a bounded artifact containing the records needed for a task, the rules determining which records are eligible, an executable verifier, and the program that performs a permitted operation. The whole can be copied, inspected, and run locally without acquiring a remote account or consulting a live model. Portability is defined operationally rather than rhetorically: after a cold transfer into an offline environment, the recipient can verify the local object, reproduce a constrained answer, and refuse altered, unauthorized, conflicting, or missing records.

A compact reference implementation accompanies this paper. Its strict mode uses a fixed exact-key grammar, canonical record envelopes, SHA-256 record hashes, a separately supplied expected root, deterministic eligibility, refusal states, and matching Python and browser JavaScript verification. A companion browser workspace demonstrates local authoring, sealed revisions, and file export. The predeclared synthetic reference suite currently passes 15 of 15 scenarios, together with 29 Python tests (including a shared cross-language boundary corpus) and 11 strict JavaScript checks. These outcomes establish bounded executable behavior in the tested environment. They do not establish external source truth, cryptographic authorship, unrestricted language understanding, cross-device compatibility on every platform, or superiority to a strong independent baseline. The proposed next experiment measures cold-start recovery, authority fidelity, refusal behavior, and dependency burden under matched tasks.

Keywords: portable knowledge, offline-first systems, evidence integrity, source authority, deterministic retrieval, browser software, reproducibility, externalized memory, dependency closure, local-first computing

1. The Question Is About Dependency, Not Storage

A knowledge project may contain everything it needs in principle while depending on services it does not control in practice. A note lives in a proprietary notebook. A rule is remembered as

a paragraph in a chat. An identity pointer resides in a provider account. A verifier exists on a remote server. The application only works while a network connection, login, subscription, API, or installed runtime remains available. Under those conditions, the information may be preserved yet its usable structure is not portable.

The proposed alternative is neither a universal rejection of networks nor a demand that all computing occur offline. It is a narrower architectural move: **make a bounded unit of work self-sufficient for its declared operations**. Information should not need to be reinterpreted from scratch each time the environment changes. The program should bring its relevant records, eligibility rules, failure behavior, and verification logic with it.

This difference becomes visible during a failure. Suppose a person has an indexed collection of technical notes and wants the most recent authorized statement about a component. If the index points to a cloud folder that cannot be reached, the answer stops there. If the notes were downloaded but the authority rules stayed inside a conversation, the recipient must guess which version controls. If the rules are present but no verifier travels with them, the recipient may unknowingly accept altered data. A complete portable unit closes these gaps for the operations it promises to support.

The guiding question is therefore not simply, "Can the files be copied?" It is: **Can the same permitted result be reconstructed, and the same forbidden result refused, after a transfer that removes the original environment?**

2. Research Position

2.1 A portable runtime is a closure of operational dependencies

Let a bounded task be T. Let M denote memory or evidence records, R the rules that govern eligibility and execution, V the validation procedure, S the executable software, and A an independently trusted anchor when authenticity relative to a known state is required. Define an operational package:

$$P_T = (M, R, V, S, A; T).$$

A package is *dependency-closed for task* T when all functions required to perform T are either inside the package or explicitly named as local environmental assumptions. Copying the package does not necessarily carry the operating system, hardware, browser, or trust in the anchor. Those dependencies must be declared. Dependency closure is always relative to the promised task, not an assertion of universal independence.

For example, an HTML document containing a local reader and a sealed set of synthetic facts can be self-contained for exact-key answers. The same file does not become a general-purpose AI assistant, a live website crawler, a secure multiuser server, or a proof of all factual claims. A local result has to be evaluated within its declared function.

2.2 Portable memory is not merely a summary

A summary can preserve what seems important while omitting version, provenance, exclusion, dissent, and revision history. A portable evidence object instead separates the stored statement from

its authorization to govern. A record may be present but retired, blocked, machine-originated, replaced by a verified successor, or in conflict with another eligible record. These distinctions can be decided by executable rules, not inferred from fluent prose.

The proposed unit therefore maintains at least four separable elements:

1. **Data:** an explicit body, identity, version, and constrained key or relationship.
2. **Policy:** the eligibility criteria and admissible operations.
3. **Verifier:** the checks that must succeed before support is granted.
4. **Reader:** the procedure that returns a bounded result or an explicit refusal.

The rule is simple: *existence is not eligibility; retrieval is not support; a valid hash is not truth*.

2.3 The network becomes optional where it is not part of the task

Local portability and network functionality are not opposites. A local corpus can be published over HTTP, shared as a ZIP, copied on removable storage, or synchronized between devices. The architectural objective is that these distribution routes should not change the rules for interpreting the same authorized state. Connection to a network is an additional capability, not the only means of opening the object.

This is the distinction between **transport dependency** and **execution dependency**. Downloading a package might require the internet once. Executing an already obtained package need not. A local browser app can run offline while its optional model integration or remote source acquisition remains unavailable. Claims of independence should name that boundary explicitly.

2.4 Related work and contribution boundary

This work builds on established practices rather than claiming to invent offline software, checksums, or portable archives. The BagIt specification [8] defines a directory-based transfer format with manifests for reliable storage and transfer. RO-Crate [9] describes research objects with structured metadata and contextual provenance. Local-first software [10] emphasizes local ownership, offline use, and continued access to data. SHA-256 follows the Secure Hash Standard [6]. These sources establish relevant design context; no empirical comparison against their implementations is reported here.

The contribution of this reference prototype is the concrete combination of a small eligibility policy, explicit refusal outcomes, a separately supplied expected-root check, two local readers, and an executable synthetic experiment. Packaging these together makes this particular operation inspectable after transfer. It does not establish that the combination is historically novel, that it replaces preservation formats, or that it outperforms established systems. A systematic related-work review and independent evaluation are still needed for a stronger research claim.

3. Architecture

3.1 The strict capsule

The reference experiment defines the format `portable-evidence-capsule/1`. Its serialization contract is deliberately order-sensitive: envelope, record-wrapper, body, and policy field order must match the fixture; records sort by ID. This is a project-specific encoding, not a claim to implement a general JSON canonicalization standard. Values contain 1–10,000 Unicode scalar values and

revisions are integers from 1 through 2^53−1, so the Python and JavaScript readers share these bounds. An envelope contains a format identifier, a fixed policy, ordered records, and a root SHA-256. Each record contains a seven-field body and its own SHA-256 digest. The body fields are `id`, `key`, `value`, `status`, `origin`, `revision`, and `supersedes`.

The policy allows answers only from records marked `current` and `human`; the retrieval grammar accepts a literal exact key and deliberately refuses paraphrase. A successful answer contains the value and evidence identifying the specific record, revision, record hash, and capsule root. A lookup returns `ambiguous` if two eligible owners match, `not_in_corpus` if no eligible owner exists, and `integrity_error` if verification fails.

This grammar is intentionally small. It avoids conflating text retrieval, semantic understanding, and factual verification. A bounded mechanism is useful precisely because a reviewer can say which claims it does and does not support.

3.2 Integrity, origin, and the trust anchor

A digest can detect that bytes have changed relative to a previously retained digest. It cannot determine whether the original statement was true or who authored it. Worse, an attacker can replace a record, recompute its digest, recompute the root, and produce a new package that is internally consistent. The hashes alone cannot reject that replacement as unauthorized.

The strict browser laboratory therefore requires an independently supplied expected root before returning a supported answer. The supplied root is displayed and can be changed by the operator; a missing or mismatched root blocks answer support. The demonstration root is pinned in the built laboratory and also distributed separately in `experiment/fixtures/demo_root.sha256`. In a real exchange, the expected root must be obtained through an independently trusted channel or replaced by an appropriate authenticated signature and trust policy. **Putting a claimed root inside the same untrusted bundle does not create authenticity.**

The reference uses SHA-256 as an integrity primitive, not a signature, public-key identity system, or historical attestation mechanism. A separate trust investigation is necessary before using this design as an authorization system. The Python and JavaScript core APIs also expose an unanchored inspection mode when no expected root is supplied; the CLI makes the root optional. Those calls check internal consistency and can return a policy-eligible value without establishing a match to an independently known state. Only the strict browser interface requires an expected root for every supported answer. The `human` origin label is declared metadata, not verified identity.

3.3 Authority and the refusal boundary

The result of an exact-key lookup is supported only if every required check succeeds. In abstract form:

$$\text{Support}(q, P) = \text{Verify}(P, A) \wedge \text{WellFormed}(q) \wedge \left|\text{EligibleOwners}(q, P)\right| = 1.$$

An eligible owner must satisfy the fixed origin and status conditions. An existing retired record is not promoted simply because a search finds it. A machine-originated record is not transformed into an authoritative human statement because it has a fluent answer. An ambiguous pair remains ambiguous until an authorized change resolves the conflict.

The supported answer is a **faithful return from a declared source object**, not an independent certification of the physical or scientific content of that object. This distinction prevents a local knowledge appliance from being mistaken for an oracle.

3.4 An authoring workspace and a strict laboratory

The accompanying archive contains two deliberately different browser surfaces.

The root `index.html` is a local authoring and retrieval workspace. It can create source and derivative objects, compute page digests, inspect pinned parent and supersession links, ask exact subject/predicate questions, and export or import a JSON ledger. Local browser storage supports convenience within the same profile. Moving data to a new device or browser profile requires explicit export and import. This mode demonstrates that a useful local source workspace can be distributed as a single HTML file. It does not establish third-party authenticity and must not be confused with the strict capsule format.

The `web/index.html` laboratory is the stricter reproducibility target. It embeds the synthetic capsule and a local JavaScript verifier, requires a matching separately supplied root to support an answer, and provides JSON import/export. An independent Python engine reads the same capsule format and verifies the same declared fixture. There is no automatic conversion between the two experimental formats in this release; they test complementary, bounded properties.

4. Hypotheses and Falsifiers

The central engineering hypothesis, H1, is that a dependency-closed package can reproduce its permitted answer and refusal behavior after being copied into a supported offline environment, without access to the originating account, remote notebook, or hosted model.

H2 is that an explicit source-governance layer will better preserve authority distinctions under adversarial fixtures than a condition in which all available text is treated as equally eligible. H3 is that separately anchored integrity verification will reject a complete internally rehashed substitution that unanchored hash checking would accept as self-consistent. H4 is that co-packaging will reduce the number of external services required for a declared task; this is a dependency count, not a claim of reduced total operating cost.

These hypotheses are falsifiable. A transferred valid capsule that cannot be opened without an undocumented remote dependency defeats H1 for the claimed platform. A retired or machine-origin record being returned as supported defeats H2. An attacker-substituted capsule being supported under an unchanged external root defeats H3. A hidden network call required for local answering defeats the narrow offline claim of H4.

Unsuccessful trials must be recorded rather than reclassified after the event. The intended evaluation distinguishes *could not open*, *opened but could not verify*, *verified but refused*, *supported correctly*, and *supported incorrectly*.

5. Experimental Design

5.1 Stage A: deterministic fixture checks

Stage A is executable in the repository now. The sample corpus is synthetic: it contains a revised kiln note, an old retired kiln note, a machine-origin draft, a blocked note, an unambiguous

permitted fact, and two conflicting human-origin records. These are artificial test records, not instructions about actual kiln operation. The corpus is built from `demo_source.json`, recorded as `demo_capsule.json`, and anchored by `demo_root.sha256`.

The declared checks include valid pinned verification; correct supported exact-key answers; exclusion of retired, machine-originated, and blocked content; refusal on ambiguity, missing keys, and natural-language requests; rejection of content, metadata, policy and root corruption; behavior after copying; rejection of complete independently rehashed substitution under the old pinned root; and refusal when the corpus is empty or missing. `scripts/run_experiment.py` writes each expected and observed status to a machine-readable result file.

The deterministic suite contains 15 cases. A separate 29-test Python suite probes schema, canonical field order, lineage, conflict, stale and rewritten data, and file relocation. Eleven JavaScript checks compare SHA-256 output and behavior with the Python-compatible fixture and check the self-contained browser artifact. The shared cross-language test covers 14 input cases, including Unicode scalar values, supplementary characters at the length boundary, invalid revisions, malformed identifiers, reordered policy fields, and mismatched anchors. This is a boundary regression corpus, not an exhaustive conformance proof. The root authoring workspace is separately built from local source files; those tests do not imply equivalence of the two formats.

5.2 Stage B: offline browser and transfer trial

The next trial uses a supported browser with network access disabled. Open the strict laboratory via `file://` from local storage. Record whether the embedded capsule verifies, whether an exact-key supported answer appears with the expected record ID, and whether unsupported prompts refuse. Turn off or deny networking before opening the file, not merely after the page has cached remote assets. Monitor outgoing requests if the test environment supports it.

Repeat with a fresh browser profile and with the HTML file on a different machine or operating system. Test an invalid anchor; a replaced JSON file with recomputed internal hashes; a damaged HTML file; and a missing expected-root record. A real transfer trial should record platform version, exact copied files, cold-start procedure, user intervention, elapsed time, network calls, success status, and any browser security restrictions.

The current repository includes a browser automation script for local Chromium testing where that tool is installed. Browser portability outside the exercised environment remains a proposed test rather than a demonstrated universal property.

5.3 Stage C: controlled comparison

A meaningful external comparison requires equally capable tasks and honest accounting for differing constraints. Four proposed conditions are:

- **C0: pointer-only handoff.** Source references and instructions exist, but the actual authoritative records remain at an account-bound or network-bound location.
- **C1: local notes.** Source content has been exported, but source-governance and verification are not executable.
- **C2: portable unanchored package.** Source content, rules, and verifier travel together; hashes are checked for internal consistency only.
- **C3: portable anchored package.** The same content, rules, and verifier travel together, with an independently retained expected root.

Compare them offline and online on the same declared question sets, including negatives and adversarial mutations. The reference implementations can exercise parts of C2 and C3 immediately. The other conditions require matched implementations and an independent trial; their outcomes must not be invented from design descriptions.

Primary outcomes are: (1) correct supported answers, (2) incorrect supported answers, (3) correct refusals, (4) wrong authority promotions, (5) successful cold transfer, and (6) external-service dependencies required per completed task. Secondary outcomes include median time to first valid result, elapsed transfer/setup time, human interventions, size of the transferred bundle, repeatability across browsers, and the cost of maintaining trust anchors or signatures. A strict result must report answerability separately from precision; blanket refusal is not a victory.

A preregistered independent test should contain unfamiliar authorized source material, human-reviewed target answers, blinded evaluation, known false leads, and at least one intentional source revision. The number of participating users, sites and test cases must be selected and justified before evaluation. Do not mistake hundreds of synthetic cases tuned alongside an implementation for representative natural-language performance.

6. Recorded Reference Results

The executable reference result recorded with this archive has the following bounded observations:

Check	Executed result	Interpretation
Declared synthetic scenario suite	15 / 15 passed	Exact expected statuses under fixed fixtures
Python tests	29 / 29 passed	Includes the 25 reference tests, three CLI tests and one shared 14-case Python/JavaScript boundary test
JavaScript strict-capsule core checks	11 / 11 passed	Local hash, retrieval, and embedded artifact behavior
JavaScript authoring-ledger checks	10 / 10 passed	Local source sealing, supersession, derivative pins, ambiguity, and export/import
Chromium offline in-memory probe	13 / 13 passed	Local HTML, fresh-context export/import, changed-data refusals, and answer refresh passed without remote requests; `file://` navigation was blocked by administrator policy
Real-world unfamiliar corpus comparison	Not conducted	No external generalization claim
Cross-device offline compatibility census	Not conducted	No universal-browser claim
Cryptographic authorship verification	Not implemented	Root trust must come from outside the bundle
Independent scientific superiority experiment	Not conducted	No causal superiority claim

The recorded results are **engineering evidence**, not a measurement of general intelligence, factual truth, authorship, security against a fully compromised host, or production reliability. The result JSON contains machine, runtime, case names, expected status, observed status, and pass/fail state. The optional browser probe demonstrates in-memory execution of the unmodified embedded HTML with Chromium network access disabled; it does not establish that the managed browser permitted direct local-file navigation. Regenerating those results is preferable to relying on a table in a paper.

The distinction between **reproducing a result** and **proving the premises of the result** remains central. The experiment can establish that a reader faithfully applies stated authority rules to controlled data. It cannot independently establish that an entered factual statement is true. A system can consistently enforce bad rules. Therefore both record provenance and rule legitimacy deserve separate review.

7. Threat Model and Limits

The reference addresses accidental change, some malicious record mutations, changed policy bytes, ambiguity, and source ineligibility. It does not address all adversarial control over the host, runtime, code, or trust anchor.

Trust-root replacement. A malicious party can change both data and hash. A separately secured expected root detects this only if the root itself is not replaced and the verifier is trusted. Signing, public-key distribution, key rotation, revocation, and trusted timestamps are outside the current bounded implementation.

Code substitution. If an attacker controls the HTML or JavaScript verifier, the displayed result may be deceptive even when a data hash appears valid. The release should itself be distributed through a trusted channel or verified using an independently known file digest or signature. A hash embedded in the attacked HTML file does not solve this problem.

Input meaning. Exact-key retrieval does not answer natural language. A keyed value may be out of context, outdated despite its declared status, or false in the world. The system does not certify source truth.

Local environment. Offline execution depends on compatible browsers, operating-system storage, and supported file APIs. Local storage can be cleared. Browser restrictions vary. Hosted pages may use a service worker or caching behavior that differs from a directly opened local file; no such behavior is required for the core standalone demos.

Transfer and privacy. A fully portable corpus may contain information that should not be shared. The ability to copy a file is not permission to distribute all content inside it. Sensitive content requires authorization, minimization, and potentially encryption, none of which the demo claims to provide.

Scale and semantics. This design does not prove large-corpus performance, arbitrary semantic matching, concurrent editing across machines, conflict-free distributed synchronization, or remote adversarial resilience. Those are separate engineering problems requiring distinct evidence.

8. Why It Matters

The engineering contribution is a shift in what is being made portable. Copying documents alone preserves information; copying documents plus rules preserves some interpretation; copying documents, rules, verification, software, and an independently held trust anchor can preserve a **reproducible operation** within declared limits.

Such an operation may be narrow, but it has a useful property: a reviewer can disconnect the network, make a cold copy, run it elsewhere, introduce controlled failures, and observe whether the same boundary holds. The work no longer depends for that bounded purpose on the continuing existence of the original notebook, conversation, account, or model provider. Cloud services may still be the best option for collaboration, updates, remote compute, or discovery. The claim is not that they must disappear. It is that they should not be silently required to interpret a self-contained body of evidence whose declared operations can be executed locally.

A portable evidence runtime also changes failure analysis. Missing data yields one result. An unknown key yields another. A record present but ineligible is not silently substituted. Multiple eligible owners produce ambiguity instead of false certainty. Hash failure prevents support. An altered but self-consistent package cannot impersonate a known baseline when the independent root is maintained. These are inspectable reasons for refusal rather than an unverifiable assurance that the software "remembers correctly."

The larger thesis is functional: **the unit of continuity should be capable of carrying enough of its own operating conditions to be checked and reused without reconstructing an absent provider environment**. When all necessary dependencies are declared, portability becomes a testable engineering property.

9. Conclusion

A memory object is not operationally independent merely because it can be downloaded. A local copy may still depend on undocumented policies, remote authorization, proprietary processing, or a reader that no longer exists. A portable evidence runtime offers a bounded alternative: package the memory, the rules that limit its authority, the verification steps that protect its known state, and the software that performs the declared operation.

The included reference experiment demonstrates deterministic offline-readable answer and refusal behavior for a synthetic capsule and a separately anchored root, together with an independent local authoring workspace. The evidence is sufficient to support the existence of a working bounded implementation in the tested environment. It is not sufficient to establish unrestricted portability, source authenticity, arbitrary understanding, or comparative scientific superiority. Those stronger claims remain open to the matched, independent transfer and authority experiments described above.

The central proposition can be stated without analogy: **if a task can be completed from a self-contained, verifiable package, remote infrastructure should be optional for that task rather than a hidden precondition of its recovery.**

References and Implementation Record

Items 1–5 document the author's implementation lineage; they are not independent evidence of effectiveness. Items 6 and 8–10 provide external technical context.

1. McPhetridge, Mitchell D. *MPLPB Local Web*. Public implementation, GitHub: https://github.com/mitchell-d00/-The-Local-Mirror-MPLPB-as-a-Self-Contained-Offline-Site.
2. McPhetridge, Mitchell D. *Smart Local MPLPB*. Public implementation, GitHub: https://github.com/mitchell-d00/SMART-LOCAL-MPLPB-A-Retrieval-Bounded-Front-End-for-a-Local-Web.
3. McPhetridge, Mitchell D. *Networked MPLPB*. Public implementation, GitHub: https://github.com/mitchell-d00/networked-mplpb.
4. McPhetridge, Mitchell D. *Combined MPLPB: A Closed Local Ledger of Pages That Answers Only From Those Pages*. Public implementation, GitHub: https://github.com/mitchell-d00/Combined-MPLPB-A-closed-local-ledger-of-pages-that-answers-only-from-those-pages.
5. McPhetridge, Mitchell D. *OneOwner, powered by MPLPB*. Public implementation, GitHub: https://github.com/mitchell-d00/OneOwner-powered-by-MPLPB.
6. National Institute of Standards and Technology. *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4.
7. *Portable Evidence Runtime* repository, this archive: `src/per/`, `web/source_runtime.js`, `experiment/fixtures/`, `scripts/run_experiment.py`, and `tests/`. Fixed, synthetic results are recorded in `experiment/results/synthetic_run.json`.
8. Kunze, J. et al. *The BagIt File Packaging Format (V1.0)*. RFC 8493, 2018. https://www.rfc-editor.org/rfc/rfc8493.
9. RO-Crate Community. *RO-Crate Specification, version 1.2*. https://www.researchobject.org/ro-crate/specification/1.2/.
10. Kleppmann, M., Wiggins, A., van Hardenberg, P., and McGranaghan, M. *Local-first software: You own your data, in spite of the cloud*. Onward!, 2019. https://doi.org/10.1145/3359591.3359737.

Repository status: Bounded reference implementation and proposed external evaluation. All publication, licensing, and release decisions remain with the author.